

Chapter One : An Overview of a Compiler

- Introduction to Compiler
- Compiler Architecture / Structure
- Phases of Compiler
- Exercise

Chapter One : An Overview of a Compiler

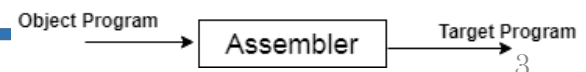
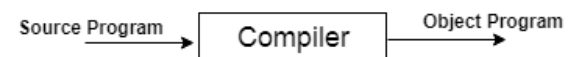
Why to Learn Compiler design?

- Computers are a balanced mix of software and hardware.
- Hardware is just a piece of mechanical device and its functions are being controlled by a compatible software.
- Hardware understands instructions in the form of electronic charge, which is the counterpart of binary language in software programming. Binary language has only two alphabets, 0 and 1.
- To instruct, the hardware codes must be written in binary format, which is simply a series of 1s and 0s.
- It would be a difficult and cumbersome task for computer programmers to write such codes, which is why we have compilers to write such codes.

Chapter One : An Overview of a Compiler

Introduction

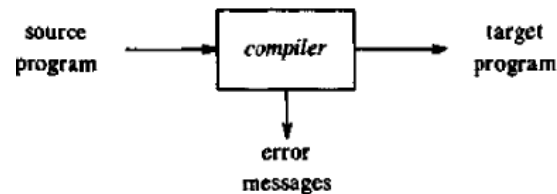
- oA compiler is a translator that converts the high-level language into the machine language.
- oHigh-level language is written by a developer and machine language can be understood by the processor.
- oCompiler is used to show errors to the programmer.
- oWhen you execute a program which is written in HLL programming language then it executes into two parts.
- oIn the first part, the source program compiled and translated into the object program (low level language).
- oIn the second part, object program translated into the target program through the assembler.



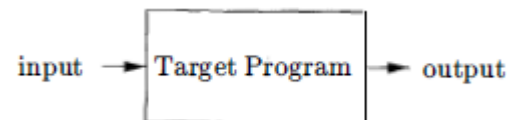
Chapter One : An Overview of a Compiler

Introduction to Compiler

- **Compiler** : Simply stated, it is a language processor , a compiler is a **program** that can read a program in one language - **the source language** - and translate it into an equivalent program in another language - **the target language**; see Fig. Below.
- An important role of the compiler is to report any errors in the source program that it detects during the translation process.



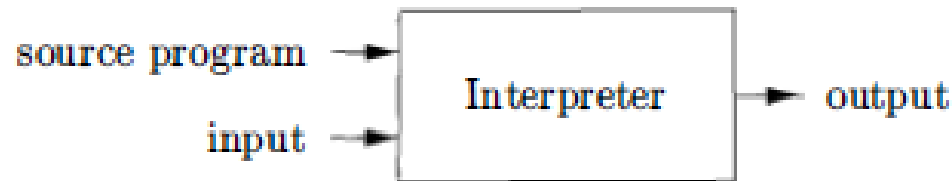
- If the target program is an executable machine-language program, it can then be called by the user to **process inputs** and **produce outputs**; see Fig. Below .



Chapter One : An Overview of a Compiler

Introduction to Compiler

- o **An interpreter** is another common kind of language processor that instead of producing a target program as a translation, an interpreter appears to directly execute the operations specified in the source program on input supplied by the user.

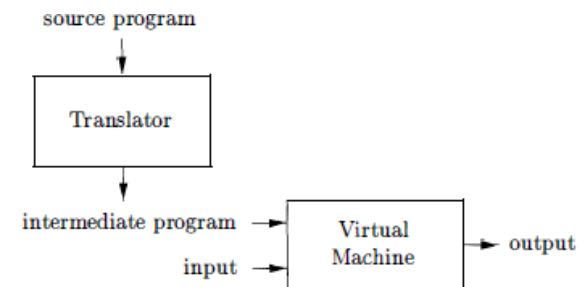


- o The machine-language target program produced by a compiler is usually much faster than an interpreter at mapping inputs to outputs. An interpreter, however, can usually give better error diagnostics than a compiler, because it executes the source program statement by statement.

Chapter One : An Overview of a Compiler

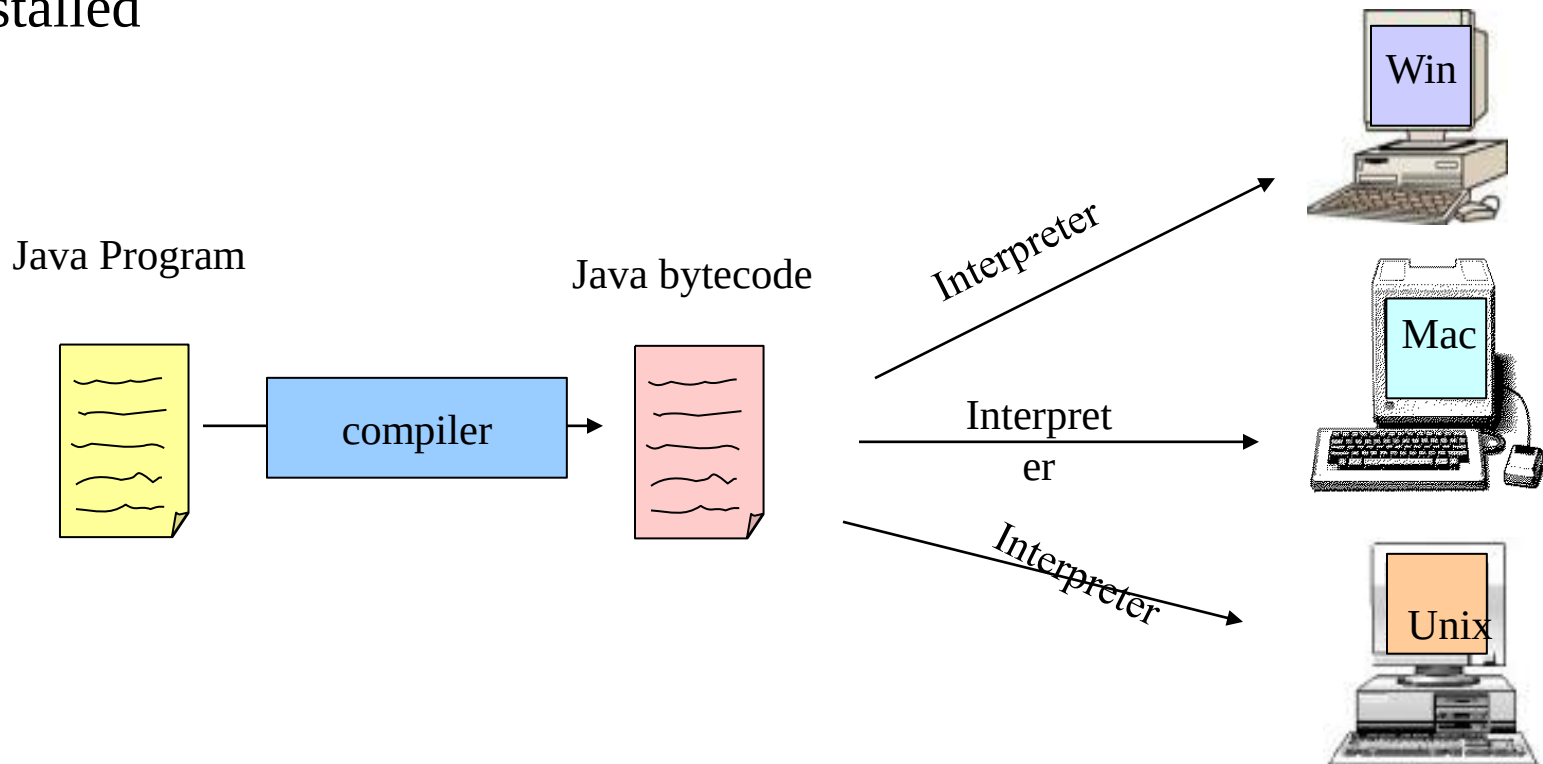
Introduction to Compiler

- o Example 1.1 : Java language processors combine compilation and interpretation, as shown in Fig. (hybrid compiler). A Java source program may first be compiled into **an intermediate form called bytecodes**. The bytecodes are then interpreted by a **virtual machine**. A benefit of this arrangement is that bytecodes compiled on one machine can be interpreted on another machine, perhaps across a network. In order to achieve faster processing of inputs to outputs, some Java compilers, called just-in-time compilers, translate the bytecodes into machine language immediately before they run the intermediate program to process the input.



E.g., Compiling Java Programs

- The Java compiler produces *bytecode* not machine code
- Bytecode is converted into machine code using a *Java Interpreter*
- You can run bytecode on any computer that has a Java Interpreter installed



Programs related to compiler...

➤ **Assemblers**

Translator for the assembly language.

Assembly code is translated into machine code

Output is relocatable machine code.

➤ **Linker**

Links object files separately compiled or assembled

Links object files to standard library functions

Generates a file that can be loaded and executed

Programs related to compiler...

➤ Loader

Loading of the executable codes, which are the outputs of linker, into main memory.

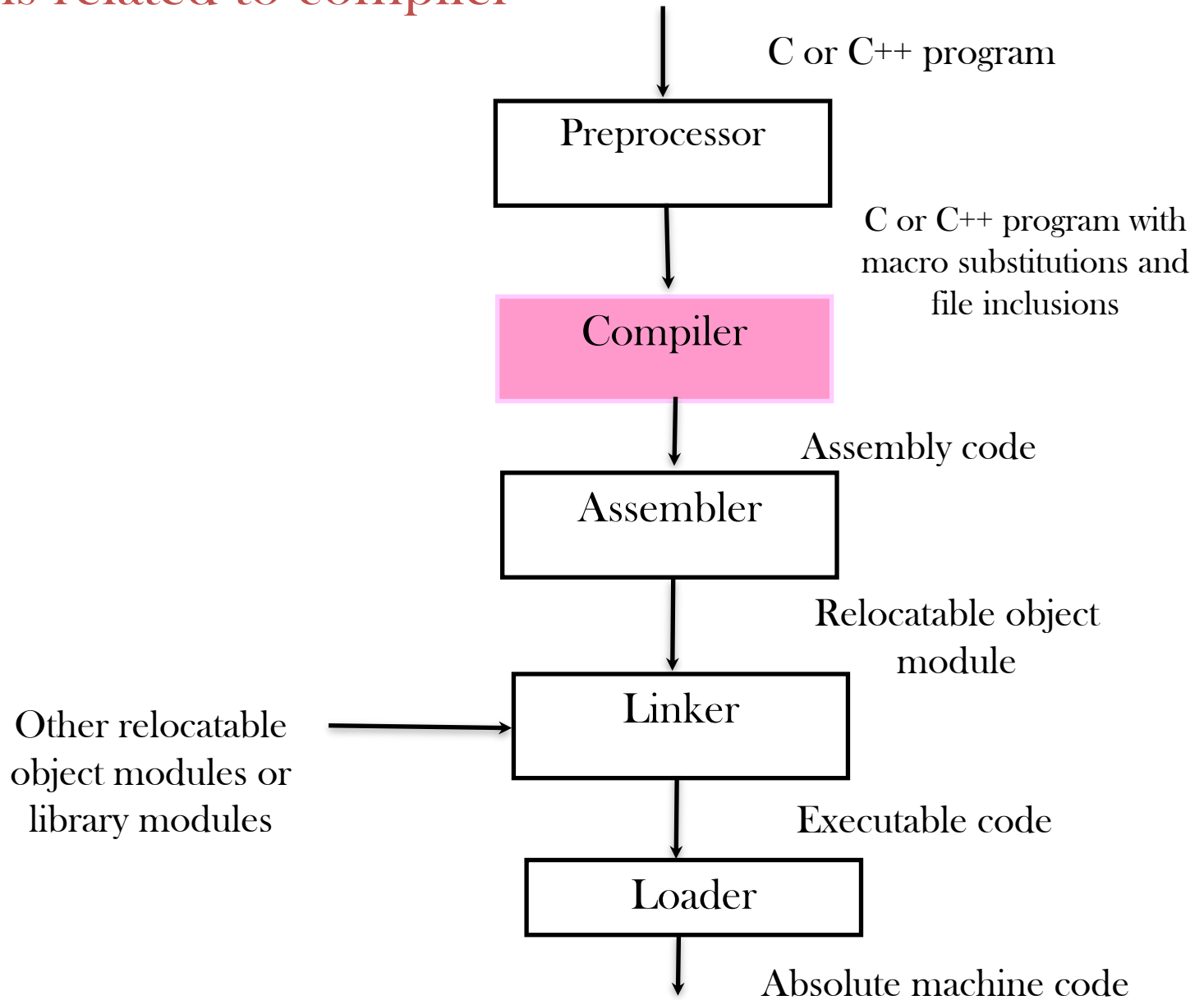
➤ Pre-processors

A pre-processor is a separate program that is called by the compiler before actual translation begins.

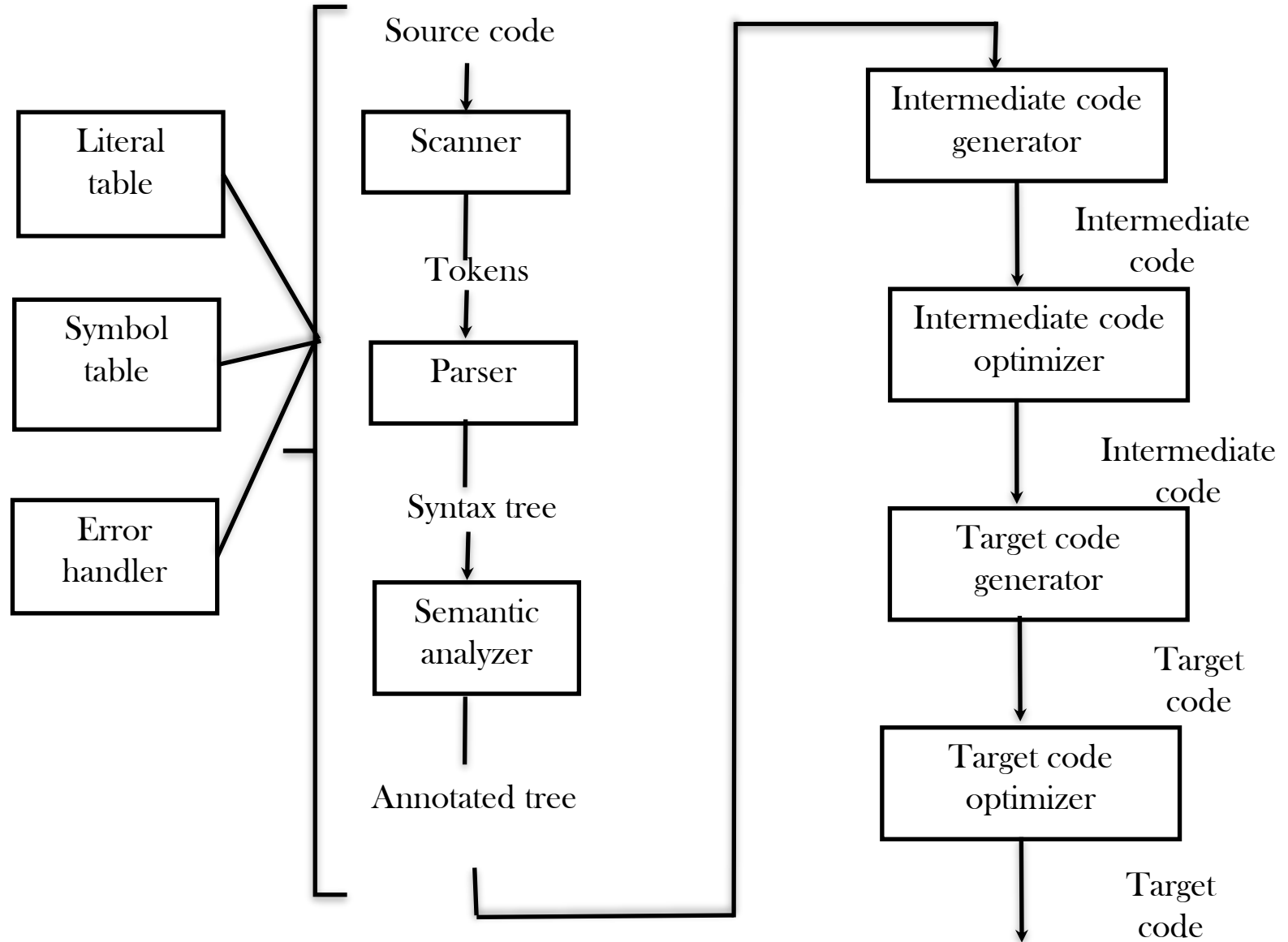
Such a pre-processor:

- Produce input to a compiler
- can delete comments,
- Macro processing (substitutions)
- include other files...

Programs related to compiler



The translation process...



Analysis and Synthesis

➤ Analysis (front end)

Breaks up the source program into constituent pieces and

Creates an intermediate representation of the source program.

During **analysis**, the operations implied by the source program are determined and recorded in hierarchical structure called a *tree*.

➤ Synthesis (back end)

The synthesis part constructs the desired program from the intermediate representation.

Analysis of the source program

Analysis consists of three phases:

Linear/Lexical analysis

Hierarchical/Syntax analysis

Semantic analysis

1. Lexical analysis or Scanning

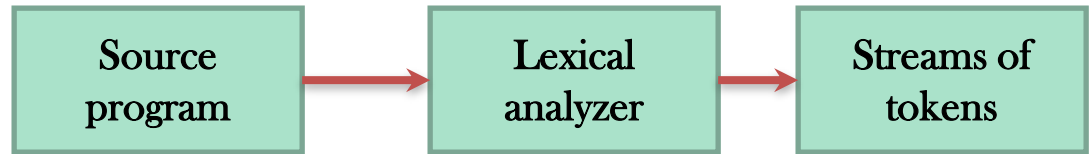
The stream of characters making up the source program is read from left to right and is grouped into *tokens*.

A **token** is a sequence of characters having a collective meaning.

A **lexical analyzer**, also called a **lexer** or a **scanner**, receives a stream of characters from the source program and groups them into tokens.

Examples:

- Identifiers
- Keywords
- Symbols (+, -, ...)
- Numbers ...



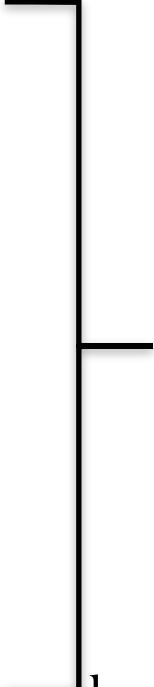
Blanks, new lines, tabulation marks will be removed during lexical analysis.

Lexical analysis or Scanning...

Example

a[index] = 4 + 2;

a	identifier
[	left bracket
index	identifier
]	right bracket
=	assignment operator
4	number
+	plus operator
2	number
;	semicolon



Tokens

A **scanner** may perform other operations along with the recognition of tokens.

- It may inter identifiers into the symbol table, and
- It may inter literals into literal table.

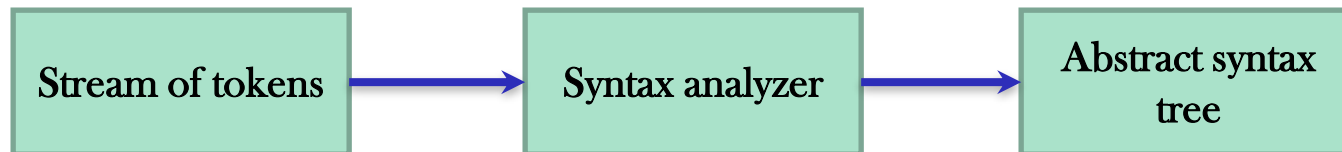
Syntax analysis or Parsing

The **parser** receives the source code in the form of **tokens** from the scanner and **performs syntax analysis**.

The results of **syntax analysis** are usually represented by a **parse tree** or a **syntax tree**.

Syntax tree each interior node represents an operation and the children of the node represent the arguments of the operation.

The syntactic structure of a programming language is determined by **context free grammar (CFG)**.



3. Semantic analysis

The semantics of a program are its **meaning** as opposed to syntax or structure

The semantics consist of:

Runtime semantics – behavior of program at runtime

Static semantics – checked by the compiler

Static semantics include:

Declarations of variables and constants before use

Calling functions that exist (predefined in a library or defined by the user)

Passing parameters properly

Type checking.

The semantic analyzer does the following:

Checks the static semantics of the language

Annotates the syntax tree with type information

Synthesis of the target program

- Intermediate code generator
- Intermediate code optimizer
- The target code generator
- The target code optimizer

Code Improvement

Code improvement techniques can be applied to:

- **Intermediate code** – independent of the target machine
- **Target code** – dependent on the target machine

Intermediate code improvement include:

- Constant folding
- Elimination of common sub-expressions
- Improving loops
- Improving function calls

Target code improvement include:

- Allocation and use of registers
- Selection of better (faster) instructions and addressing modes.

Intermediate code generator

Comes after syntax and semantic analysis

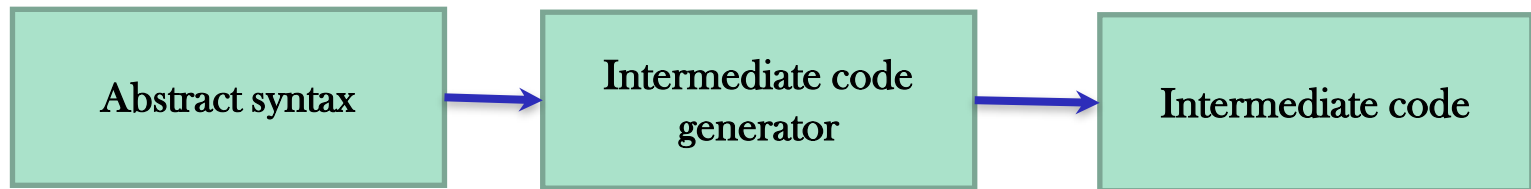
Separates the compiler front end from its backend
Intermediate representation should have 2 important properties:

- Should be easy to produce

- Should be easy to translate into the target program

Intermediate representation can have a variety of forms:

Three-address code, P-code for an abstract machine, Tree or DAG representation



□ Three address code for the original C expression **a[index]=4+2** is:

$$\begin{aligned}t_1 &= 2 \\ t_2 &= 4 + t_1 \\ \mathbf{a[index]} &= t_2\end{aligned}$$

IC optimizer

- An **IC optimizer** reviews the code, looking for ways to reduce:
 - m** the number of operations and
 - m** the memory requirements.

A program may be optimized for speed or for size.

- This phase changes the IC so that the code generator produces a **faster** and **less memory** consuming program.
- The optimized code does the same thing as the original (non-optimized) code but with less cost in terms of CPU time and memory space.



IC optimizer...

There are several techniques of optimizing code and they will be discussed in the forthcoming chapters.

Ex. Unnecessary lines of code in loops (i.e. code that could be executed outside of the loop) are moved out of the loop.

```
for(i=1; i<10; i++){
```

```
  x = y+1;
```

```
  z = x+i; }
```



```
x = y+1;  
for(i=1; i<10, i++)  
  z = x+i;
```

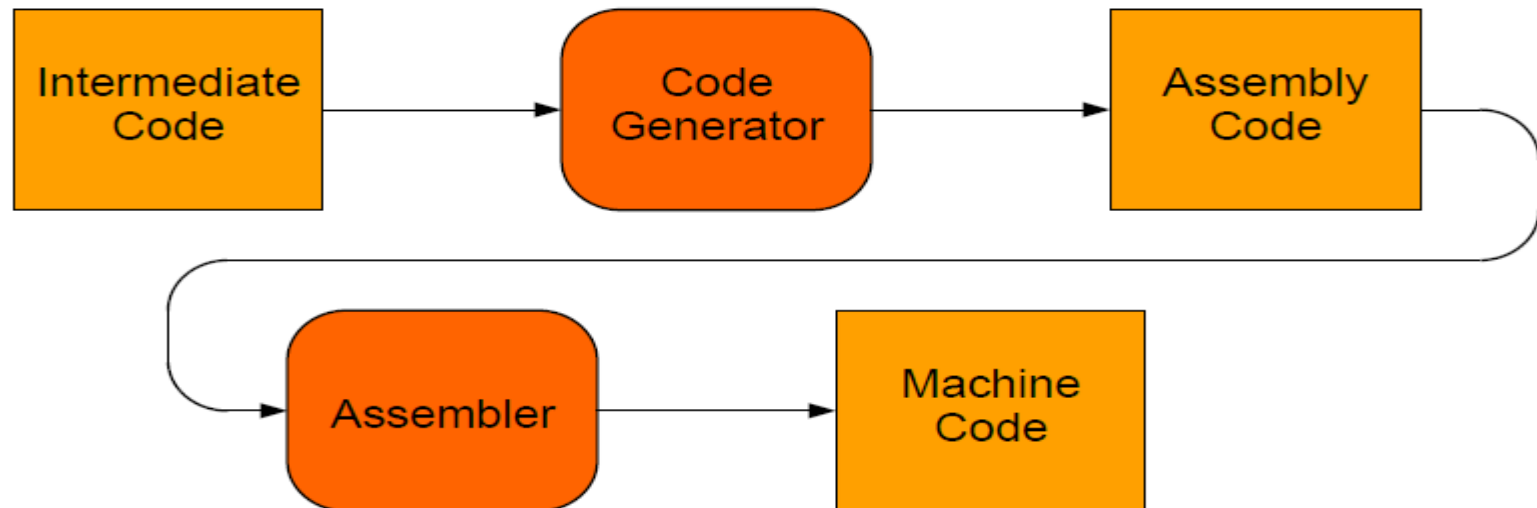
Code generator

The **machine code generator** receives the (optimized) intermediate code, and then it produces either:

- **Machine code** for a specific machine, or
- **Assembly code** for a specific machine and assembler.

Code generator

- Selects appropriate machine instructions
- Allocates memory locations for variables
- Allocates registers for intermediate computations



The target code optimizer

In this phase, the compiler attempts to improve the target code generated by the code generator.

Such improvement includes:

- Choosing addressing modes to improve performance
- Replacing slow instruction by faster ones
- Eliminating redundant or unnecessary operations

In the sample target code given, use a shift instruction to replace the multiplication in the second instruction.

Another is to use a more powerful addressing mode, such as indexed addressing to perform the array store.

Chapter One : An Overview of a Compiler

Exercises

- What is the difference between a compiler and an interpreter?
- What are the advantages of (a) a compiler over an interpreter
(b) an interpreter over a compiler?
- What advantages are there to a language-processing system in which the compiler produces assembly language rather than machine language?
- A compiler that translates a high-level language into another high-level language is called a source-to-source translator. What advantages are there to using C as a target language for a compiler?
- Describe some of the tasks that an assembler needs to perform.